



07. 객체 수명, RAII와 소유권

Heesung Oh

<https://github.com/3dapi>



- 저장 기간과 객체 수명의 구분
- 동적 메모리와 자원 관리 문제
- **RAII** 와 결정적 소멸
- 예외 처리와 스택 되감기
- 예외 안전성의 보장 수준
- **std::unique_ptr** 와 고유 소유권
- **std::shared_ptr** 와 공유 소유권
- **std::weak_ptr** 와 비소유 관찰
- 값, 참조, 포인터와 스마트 포인터 선택

객체 생성
↓
자원 획득
↓
객체 사용
↓
객체 소멸
↓
자원 해제

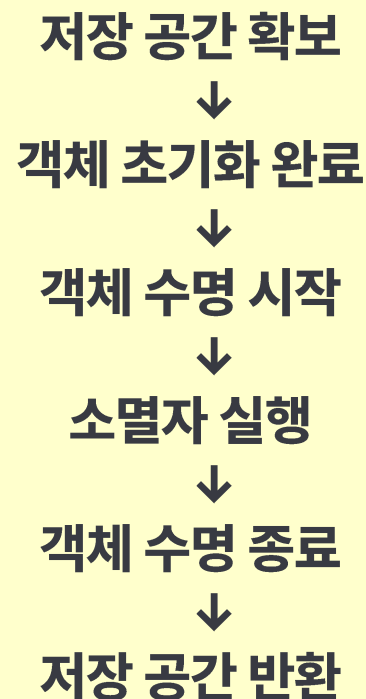




7.1 저장 기간과 객체 수명

- 저장 기간: 객체가 사용할 저장 공간이 유지되는 기간
- 객체 수명: 초기화된 객체를 해당 타입으로 사용할 수 있는 기간
- 저장 공간이 있어도 객체 수명이 시작되지 않았으면 사용할 수 없음
- 객체가 소멸한 뒤 주소가 남아 있어도 사용할 수 없음

구분	의미	기준
저장 기간	저장 공간 유지 기간	선언 위치와 할당 방식
객체 수명	타입 객체로 사용 가능한 기간	초기화 완료와 소멸 완료





- 함수나 블록 안의 일반 지역 객체
- 선언 지점에서 초기화
- 블록을 벗어날 때 소멸
- 같은 블록에서는 생성 역순으로 소멸

```
void TestScope()  
{  
    TraceObject first("first");  
    TraceObject second("second");  
}
```

first 생성
second 생성
second 소멸
first 소멸

```
void Update()  
{  
    TraceObject outer("outer");  
    {  
        TraceObject inner("inner");  
    }  
    std::cout << "Update 계속\n";  
}
```

- 좁은 블록을 사용하면 자원을 더 일찍 해제 가능



- 이름 공간 범위 객체
- static 지역 객체
- 프로그램 실행 동안 저장 공간 유지
- 정적 지역 객체는 처음 실행이 선언 지점에 도달할 때 초기화

```
int totalScore = 0;
int NextId()
{
    static int id = 0;
    return ++id;
}
GameConfig& GetConfig()
{
    static GameConfig config;
    return config;
}
```

객체	초기화 시점	소멸 시점
전역 객체	일반적으로 main() 전	프로그램 종료 과정
정적 지역 객체	최초 실행 도달 시점	프로그램 종료 과정

- 서로 다른 소스 파일의 정적 객체 초기화 순서 의존 주의



- 실행 중 명시적으로 저장 공간 확보
- **new** 로 객체 생성
- **delete** 로 객체 수명 종료와 저장 공간 반환
- 생성한 블록을 벗어나도 자동 소멸하지 않음

```
int* value = new int(100);  
std::cout << *value << '\n';  
delete value;
```

```
Enemy* CreateEnemy()  
{  
    Enemy* enemy = new Enemy;  
    return enemy;  
}
```

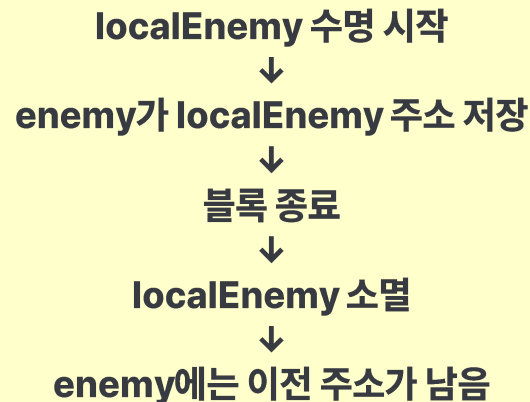
필요 상황	예
함수보다 긴 수명	장면에 등록된 객체
실행 중 개수 결정	적 목록, 리소스 목록
소유권 이전	팩토리 함수
다형 객체 관리	기반 클래스 포인터

- 원소 집합은 **std::vector** 우선
- 단일 동적 객체는 **스마트 포인터** 우선



```
Enemy* enemy = nullptr;  
{  
    Enemy localEnemy;  
    enemy = &localEnemy;  
}
```

// enemy->Update(); // 잘못된 접근



- 포인터 값이 nullptr가 아니어도 객체가 살아 있다는 뜻은 아님
- 소멸한 객체를 가리키는 포인터는 댕글링 포인터
- 참조도 대상 객체 수명보다 오래 사용하면 같은 문제 발생

수명 구분이 필요한 이유

- 지역 객체 주소 반환 금지
- delete 이후 포인터 사용 금지
- 컨테이너 재배포 후 기존 포인터 무효화 가능
- 예외 발생 시 생성 완료 객체만 소멸



- new와 delete는 동적 객체의 생성·소멸 원리 이해에 필요
- 실제 프로그램에서 원시 포인터만으로 소유권 표현은 위험
- 해제 책임이 인터페이스에 드러나지 않음

```
Resource* CreateResource(); // 호출자가 delete해야 하는가?  
Resource* FindResource(int id); // 내부 객체를 빌려주는가?
```

원시 포인터

- └ 소유 포인터일 수 있음
- └ 비소유 포인터일 수 있음
- └ 타입만 보고 구분 어려움

문제	결과
해제 누락	메모리 누수
중복 해제	이중 해제
소멸 뒤 접근	댕글링 포인터
잘못된 해제 방식	미정의 동작

- RAII와 스마트 포인터로 같은 문제 해결





7.2.1 new와 delete

- new는 저장 공간 확보와 객체 초기화를 함께 수행
- delete는 소멸자 호출 후 저장 공간 반환
- delete 뒤 포인터 변수는 자동으로 nullptr가 되지 않음

```
int* first = new int;  
int* second = new int(100);  
int* third = new int{};
```

```
delete first;  
delete second;  
delete third;
```

표현식	초기화
new int	기본 초기화, 값 불명
new int(100)	100으로 초기화
new int{}	0으로 값 초기화

```
Resource* resource = new Resource(1);  
delete resource;  
resource = nullptr;
```

- delete nullptr;은 안전
- 같은 객체에 두 번 delete는 미정의 동작



7.2.2 new [] 와 delete []

- 동적 배열은 **new []** 로 생성, **delete []** 로 해제
- new [] 와 delete 를 섞으면 미정의 동작

```
std::size_t count = 5;  
int* values = new int[count]{};  
  
for (std::size_t i = 0; i < count; ++i)  
{  
    values[i] = static_cast<int>(i * 10);  
}  
delete[] values;  
  
Resource* resources = new Resource[3]{  
    Resource(1),  
    Resource(2),  
    Resource(3)  
};  
delete[] resources;
```

생성	해제
new T	delete
new T[n]	delete[]

- 원시 포인터 타입에는 배열 크기 정보가 없음
- 일반적인 가변 길이 배열은 **std::vector** 권장



- 동적으로 확보한 메모리를 해제하지 못한 상태
- 포인터 변수만 사라지고 동적 객체는 남을 수 있음
- 주소를 잃으면 정상적인 해제 불가

```
void Leak()
{
    int* value = new int(100);
    // delete value; 없음
}

int* value = new int(10);
value = new int(20); // 첫 번째 주소를 잃음
delete value;

bool Process(bool valid)
{
    int* values = new int[100]{};

    if (!valid)
    {
        return false; // values 누수
    }

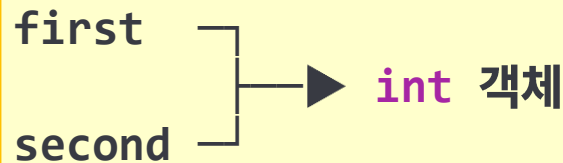
    delete[] values;
    return true;
}
```

정상 종료 경로만 해제
↓
조기 **return** 또는 예외
↓
해제 코드 건너뛰림



7.2.5 이중 해제와 댕글링 포인터

```
int* first = new int(100);  
int* second = first;
```



```
delete first;  
first = nullptr;
```

```
// std::cout << *second; // 댕글링 포인터 역참조  
// delete second;      // 이중 해제
```

- 두 포인터가 있어도 객체는 하나
- 한 포인터로 해제하면 다른 포인터도 유효하지 않음
- 한 포인터를 `nullptr`로 바꾸어도 다른 별칭 포인터는 바뀌지 않음

```
const std::string& GetNameReference()  
{  
    std::string name = "Player";  
    return name; // 잘못된 반환  
}
```

- 지역 객체의 주소나 참조 반환 금지



- 메모리 이외 프로그램이 관리해야 하는 자원 다수
- 획득과 해제 함수가 쌍으로 존재
- 모든 종료 경로에서 정확히 한 번 해제

```
void SaveData(bool valid)
{
    std::FILE* file = std::fopen("data.txt", "w");
    if (file == nullptr)
    {
        return;
    }
    if (!valid)
    {
        std::fclose(file);
        return;
    }
    std::fputs("Player\n", file);
    std::fclose(file);
}
```

- 반환 경로가 늘수록 정리 코드 누락 위험 증가

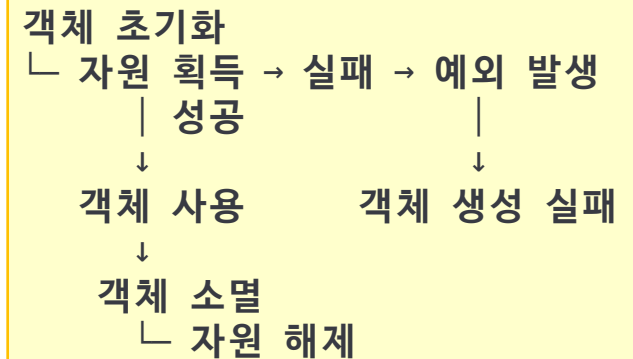
자원	획득	해제
파일	열기	닫기
소켓	생성·연결	종료·닫기
스레드	시작	결합 또는 분리
무텍스	잠금	잠금 해제
그래픽 자원	버퍼·텍스처 생성	API 해제 함수
운영체제 핸들	API 호출	닫기 함수



- Resource Acquisition Is Initialization
- 자원 획득을 객체 초기화와 연결
- 자원 해제를 객체 소멸과 연결
- 정상 종료, 조기 반환, 예외 경로에서 같은 정리 규칙 적용

```
void Process()
{
    std::unique_ptr<int> value =
        std::make_unique<int>(100);

    // 범위를 벗어나면 자동으로 메모리 해제
}
```



RAII 대상

- 동적 메모리
- 파일
- 소켓
- 잠금
- 그래픽 리소스
- 운영체제 핸들
- 데이터베이스 연결



7.3.1 자원 획득과 객체 초기화

- RAII 객체는 생성 완료 후 유효한 자원을 소유하도록 구성
- 자원 획득 실패 시 생성 실패를 알림
- 완성되지 않은 객체를 사용하게 만들지 않음

```
class FileHandle
{
public:
    FileHandle(const char* fileName, const char* mode)
        : file(std::fopen(fileName, mode))
    {
        if (file == nullptr)
        {
            throw std::runtime_error("파일을 열 수 없습니다.");
        }
    }
    ~FileHandle()
    {
        std::fclose(file);
    }
private:
    std::FILE* file = nullptr;
};
```

```
void SaveData()
{
    FileHandle file("data.txt", "w");
}
```

- 생성 완료된 FileHandle은 유효한 파일을 소유한다는 불변식 유지



- 원시 자원을 클래스 안에 저장
- 소멸자에서 자원 해제
- 복사 금지로 중복 소유 방지
- 이동으로 소유권 이전

```
class FileHandle
{
public:
    FileHandle(const char* fileName, const char* mode);
    ~FileHandle();

    FileHandle(const FileHandle&) = delete;
    FileHandle& operator=(const FileHandle&) = delete;

    FileHandle(FileHandle&& other) noexcept
        : file(other.file)
    {
        other.file = nullptr;
    }

    FileHandle& operator=(FileHandle&& other) noexcept
    {
        if (this == &other)
        {
            return *this;
        }
        Close();
        file = other.file;
        other.file = nullptr;
        return *this;
    }
private:
    void Close() noexcept;
    std::FILE* file = nullptr;
};
```




7.3.5 Rule of Zero와 RAII

- 직접 자원 관리 코드는 작은 래퍼에 한정
- 상위 클래스는 RAII 타입을 값 멤버로 사용
- 특별 멤버 함수를 직접 작성하지 않아도 되는 구조 지향
- Rule of Zero는 자원 관리를 검증된 타입에 맡기는 설계 원칙

```
class SaveSystem
{
public:
    explicit SaveSystem(const std::string& path)
        : path(path){ }
private:
    std::string path;
    std::vector<std::string> pendingFiles;
};

class GameScene
{
private:
    std::vector<std::unique_ptr<Enemy>> enemies;
};
```

저수준 자원 래퍼
FileHandle, **unique_ptr**, **vector**, **string**
↓
상위 업무 객체
SaveSystem, GameScene, Player



- 함수가 정상 결과를 만들 수 없을 때 오류 정보를 전달, throw 로 예외 발생
- try 블록에서 실행, catch 블록에서 처리
- 처리할 수 있는 지점까지 호출 경로를 거슬러 올라감

```
int Divide(int left, int right)
{
    if (right == 0)
    {
        throw std::invalid_argument("0으로 나눌 수 없습니다.");
    }
    return left / right;
}

try {
    int result = Divide(10, 0);
    std::cout << result << '\n';
}
catch (const std::invalid_argument& error)
{
    std::cerr << error.what() << '\n';
}
```

예외 객체는 값으로 던지고
const 참조로 받는 방식이 일반적



예외 타입	대표 의미
<code>std::exception</code>	표준 예외 계층 공통 기반
<code>std::logic_error</code>	논리 오류, 사전 조건 오류
<code>std::invalid_argument</code>	함수 인수가 유효하지 않음
<code>std::out_of_range</code>	인덱스나 값이 범위를 벗어남
<code>std::length_error</code>	요청 크기가 허용 범위 초과
<code>std::runtime_error</code>	실행 환경 때문에 작업 실패
<code>std::bad_alloc</code>	동적 메모리 할당 실패

```
int& FixedArray::At(std::size_t index)
{
    if (index >= size)
    {
        throw std::out_of_range("인덱스 범위 초과");
    }
    return values[index];
}
if (file == nullptr)
{
    throw std::runtime_error("파일 열기 실패");
}
```

표준 예외로 의미를 충분히 표현할 수 있으면
새 예외 타입이 필요하지 않음



7.4.4 스택 되감기

- 예외가 발생하면 일치하는 catch를 찾을 때까지 호출 경로를 거슬러 올라감
- 이 과정에서 생성 완료된 자동 객체가 역순으로 소멸
- RAII 객체는 스택 되감기 중 자원을 정리

```
void LoadLevel()
{
    TraceObject first("first");
    FileHandle file("level.dat", "rb");
    TraceObject second("second");
    throw std::runtime_error("레벨 로드 실패");
}
```

first 생성
파일 열기
second 생성
예외 발생
second 소멸
파일 닫기
first 소멸
catch 실행

```
void LoadLevel()
{
    std::unique_ptr<Resource> resource =
        std::make_unique<Resource>(1);

    throw std::runtime_error("실패");
}
```

- 원시 포인터가 가리키는 동적 객체는 자동 소멸하지 않음



- 생성자가 유효한 객체를 만들 수 없으면 예외 발생 가능
- 생성자가 예외로 끝나면 완성된 객체가 만들어지지 않음
- 완성되지 않은 객체 자신의 소멸자는 호출되지 않음
- 이미 생성된 멤버 객체는 자동으로 소멸

```
class LevelPackage
{
public:
    LevelPackage(): config("config.txt", "rb"),
                    map("missing.map", "rb")
    {}
    ~LevelPackage()
    {
        std::cout << "LevelPackage 소멸\n";
    }
private:
    FileHandle config; FileHandle map;
};
```

config 생성 완료
map 생성 실패
config 소멸
LevelPackage 객체 생성 실패
LevelPackage 소멸자는 호출되지 않음

- 멤버를 RAII 타입으로 구성하면 생성 중 실패한 앞쪽 자원도 정리



- 소멸자는 예외를 밖으로 전달하지 않도록 설계
- 스택 되감기 중 소멸자에서 예외가 나오면 `std::terminate()` 가능
- 실패를 호출자가 알아야 하는 작업은 명시적 함수로 분리

```
class Resource
{
public:
    ~Resource() noexcept
    {
        Release();
    }
};
```

- 소멸자는 가능한 범위의 최종 정리 수행
- 오류 보고가 필요하면 `Commit()`, `Close()`, `Flush()` 사용

```
class OutputFile
{
public:
    void Commit()
    {
        if (!FlushData()) {
            throw std::runtime_error("파일 저장 실패");
        }
        committed = true;
    }
    ~OutputFile() noexcept {
        if (!committed) {
            TryDiscardWithoutThrowing();
        }
    }
private:
    bool committed = false;
};
```



- 예외가 발생해도 자원이 누수되지 않고 객체 상태가 유효해야 함
- 예외를 발생시키지 않는다는 뜻이 아님
- 실패 가능한 연산의 순서와 상태 표현을 설계하는 문제

```
class Inventory
{
public:
    void AddItem(const std::string& item)
    {
        ++itemCount;
        items.push_back(item);
    }
private:
    int itemCount = 0;
    std::vector<std::string> items;
};
```

- push_back() 실패 시 itemCount와 실제 원소 수 불일치

```
void AddItem(const std::string& item)
{
    items.push_back(item);
    itemCount =
static_cast<int>(items.size());
}
```

- 실패 전에는 itemCount를 바꾸지 않음
- 중복 상태를 제거하면 더 단순

```
std::size_t ItemCount() const
{
    return items.size();
}
```



7.5 예외 안전성 보장 수준

보장 수준	예외 발생 후 상태
보장 없음	자원 누수나 불변식 손상 가능
기본 보장	자원 누수 없음, 객체는 유효하지만 값은 변경될 수 있음
강한 보장	연산 전 상태 유지
무 예외 보장	함수가 예외를 발생시키지 않음

기본 보장
성공한 일부 변경은 남을 수 있음
객체는 계속 사용 가능

강한 보장
성공 → 새 상태 반영
실패 → 이전 상태 유지

무 예외 보장
예외가 밖으로 전달되지 않음
소멸자, 해제 함수, 단순 이동에 중요

- 모든 함수가 강한 보장을 제공해야 하는 것은 아님
- 비용, 외부 효과, 롤백 가능성을 함께 고려





- 원본의 복사본을 먼저 생성
- 복사본 생성 성공 후 현재 객체와 교환
- 임시 복사본이 이전 자원을 소멸
- Swap()이 무 예외 이면 강한 보장 구현이 쉬움

```
IntArray& IntArray::operator=(const IntArray&
other)
{
    if (this == &other)
    {
        return *this;
    }
    IntArray copy(other);
    Swap(copy);
    return *this;
}

void IntArray::Swap(IntArray& other) noexcept
{
    std::swap(size, other.size);
    std::swap(data, other.data);
}
```

복사본 생성 실패 → 현재 객체 유지
복사본 생성 성공 → swap
임시 복사본 소멸 → 이전 자원 정리

- 구현은 단순하지만
기존 버퍼 재사용이 어려울 수 있음



- 함수가 예외를 밖으로 전달하지 않는다는 계약
- 실제로 지킬 수 있는 함수에만 지정
- 예외가 noexcept 함수 밖으로 나오면 `std::terminate()` 호출

```
FileHandle(FileHandle&& other) noexcept;  
FileHandle& operator=(FileHandle&& other) noexcept;  
void Swap(FileHandle& other) noexcept;
```

지정 후보

- 소멸자와 자원 해제 함수
- 단순 이동 생성자와 이동 대입 연산자
- 포인터나 기본형 멤버만 교환하는 `Swap()`
- 실패 가능성이 없는 조회 함수
- 컨테이너 재배치에 활용되는 이동 연산

noexcept



성능 장식이 아니라 예외 안전성 계약

```
void LoadFile(const std::string& fileName); // 실패 가능
```



7.6 std::unique_ptr

- 동적 객체 하나에 대한 고유 소유권 표현
- 하나의 unique_ptr만 객체를 소유
- 소유자가 소멸하면 동적 객체도 소멸
- 복사 불가, 이동 가능

```
#include <memory>
std::unique_ptr<Enemy> enemy = std::make_unique<Enemy>();
enemy->Update();
void Play()
{
    auto enemy = std::make_unique<Enemy>();
    enemy->Update();
} // Enemy 자동 소멸
```

unique_ptr —► Enemy
소유자 1개

적합한 상황

- 객체 수명이 현재 범위보다 길어질 수 있음
- 소유권을 다른 객체나 함수로 이전
- 실행 중 실제 타입 결정
- 객체가 선택적으로 존재하여 빈 상태 필요
- 큰 객체의 주소 안정성 필요





7.6.2 std::make_unique

- C++14부터 제공
- 객체 생성과 스마트 포인터 생성을 함께 표현
- 원시 포인터가 중간에 드러나는 것을 줄임
- new와 타입 이름 반복 감소

```
auto enemy = std::make_unique<Enemy>(100, 20);  
class Enemy  
{  
public:  
    Enemy(int hp, int attack) : hp(hp), attack(attack)  
    { }  
private:  
    int hp;  
    int attack;  
};  
std::unique_ptr<Enemy> enemy(new Enemy(100, 20));
```

- 일반적으로 make_unique() 우선

```
auto value = std::make_unique<int>(); // int 값 초기화, 0
```

- 사용자 정의 삭제자가 필요한 경우에는 생성자 직접 사용 가능



7.6.3 이동 전용 스마트 포인터

- 고유 소유권은 동시에 둘 이상 존재할 수 없음
- `unique_ptr`는 복사 불가
- 소유권은 이동으로 이전

```
auto first = std::make_unique<Enemy>();  
// auto second = first; // 오류
```

```
auto second = std::move(first);
```

first —————> nullptr
second —————> Enemy

```
if (second)  
{  
    second->Update();  
}  
first = std::make_unique<Enemy>();
```

```
class GameScene  
{  
private:  
    std::unique_ptr<Enemy> boss;  
};
```

- `unique_ptr` 멤버가 있으면 클래스 기본 복사도 사용할 수 없게 됨
- 장면 복사가 필요하면 깊은 복사, 공유 리소스 분리, 복사 금지, `Clone()` 중 선택



- 동적 배열의 고유 소유권은 `std::unique_ptr<T[]>`
- 소멸 시 `delete[]` 사용
- 단일 객체용과 배열용이 타입에서 구분

```
std::unique_ptr<int[]> values =  
std::make_unique<int[]>(10);  
  
for (std::size_t i = 0; i < 10; ++i)  
{  
    values[i] = static_cast<int>(i * 10);  
}  
  
std::unique_ptr<int> one =  
std::make_unique<int>(100);  
std::unique_ptr<int[]> many =  
std::make_unique<int[]>(100);
```

```
class ImageBuffer  
{  
public:  
    ImageBuffer(int width, int height)  
        : width(width), height(height),  
          pixels(std::make_unique<unsigned int[]>(  
              static_cast<std::size_t>(width) * height))  
    {}  
private:  
    int width, height;  
    std::unique_ptr<unsigned int[]> pixels;  
};
```

- 배열 크기는 `unique_ptr<T[]>`에 저장되지 않음
- 일반 컨테이너 기능이 필요하면 `std::vector<T>` 우선



7.6.5 get, reset, release

함수	객체 소멸	unique_ptr 소유권	반환값
get()	하지 않음	유지	비소유 원시 포인터
reset()	기존 객체 소멸	비우거나 새 객체 소유	없음
release()	하지 않음	포기	해제 책임이 있는 원시 포인터

```
auto enemy = std::make_unique<Enemy>();  
Enemy* pointer = enemy.get(); // 비소유  
UseEnemy(pointer);  
  
enemy.reset();  
enemy = std::make_unique<Enemy>(200, 30);  
  
Enemy* raw = enemy.release();  
// enemy는 빈 상태  
  
delete raw;  
  
// enemy.release(); // 반환 포인터를 잃으면 누수
```

- 객체 접근은 **get()**
- 소유권 포기는 **release()**를 제한적으로 사용



7.6.7 unique_ptr와 함수 인터페이스

인터페이스	의미
<code>std::unique_ptr<T> Create()</code>	새 고유 소유권 반환
<code>void Take(std::unique_ptr<T>)</code>	호출자로부터 고유 소유권 받음
<code>void Replace(std::unique_ptr<T>&)</code>	호출자의 스마트 포인터 자체 변경 가능
<code>void Use(T&)</code>	객체를 변경 가능하게 빌림
<code>void Read(const T&)</code>	객체를 읽기 전용으로 빌림
<code>void Observe(T*)</code>	없을 수도 있는 비소유 접근

```
std::unique_ptr<Enemy> CreateEnemy(int hp, int attack)
{
    return std::make_unique<Enemy>(hp, attack);
}
void GameScene::AddEnemy(std::unique_ptr<Enemy> enemy)
{
    enemies.push_back(std::move(enemy));
}
```

```
auto enemy = CreateEnemy(100, 20);
scene.AddEnemy(std::move(enemy));
```

- 소유하지 않고 사용만 하면 `Enemy&`, `const Enemy&`, `Enemy*` 사용

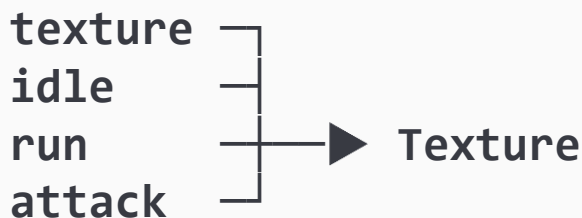


7.7 std::shared_ptr

- 여러 스마트 포인터가 하나의 동적 객체를 공동 소유
- 마지막 소유자가 사라질 때 객체 소멸
- 복사하면 객체 복사가 아니라 소유권 공유
- 마지막 사용자를 미리 특정하기 어려운 공유 수명에 적합

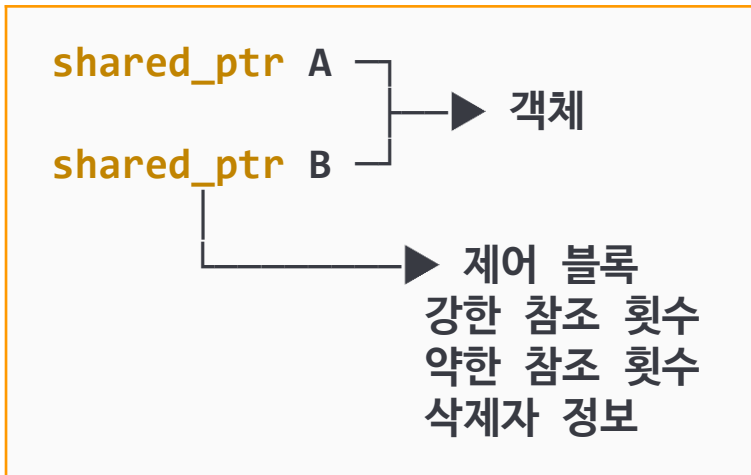
```
class Sprite
{
public:
    explicit Sprite(std::shared_ptr<Texture> texture)
        : texture(std::move(texture)) { }
private:
    std::shared_ptr<Texture> texture;
};
auto texture =
std::make_shared<Texture>("character.png");
```

```
Sprite idle(texture);
Sprite run(texture);
Sprite attack(texture);
```





- `shared_ptr`는 객체 포인터와 제어 블록을 사용
- 제어 블록은 강한 참조 횟수, 약한 참조 횟수, 삭제자 정보 관리



```
auto first = std::make_shared<Texture>("hero.png");
auto second = first;
std::cout << first.use_count() << '\n';
```

- `shared_ptr` 복사 → 강한 참조 횟수 증가
- 소유자 해제 → 강한 참조 횟수 감소
- 강한 참조 횟수 0 → 관리 대상 객체 소멸
- `weak_ptr`가 남아 있으면 제어 블록은 더 오래 남을 수 있음
- `use_count()`를 핵심 로직 조건으로 사용하지 않음



- make_shared()는 객체와 제어 블록을 함께 생성 가능
- 생성 비용과 예외 안전성 측면에서 유리
- 원시 포인터가 중간에 노출되지 않음

```
auto texture = std::make_shared<Texture>("hero.png");  
std::shared_ptr<Texture> texture(new Texture("hero.png"));
```

공유 소유권의 비용

- 제어 블록 메모리
- 복사와 소멸 시 참조 횟수 변경
- 동시 실행을 고려한 참조 횟수 조작
- 실제 소유자 불분명
- 순환 참조 가능성
- 객체 수명이 예상보다 길어질 가능성

여러 객체가 사용한다 ≠ 여러 객체가 수명을 함께 책임진다

- 단순 사용 관계라면 고유 소유자 하나와 비소유 참조 여러 개 검토



7.7.5 잘못된 shared_ptr 생성

- 같은 원시 포인터로 독립된 shared_ptr를 만들면 안 됨
- 서로 다른 제어 블록이 같은 객체를 소유한다고 판단
- 마지막 소유자 판정이 두 번 발생
- 이중 해제 가능

```
Texture* raw = new Texture("hero.png");  
  
std::shared_ptr<Texture> first(raw);  
std::shared_ptr<Texture> second(raw); // 잘못된 생성
```

first —▶ 제어 블록 A —▶ Texture
second —▶ 제어 블록 B —▶ 같은 Texture

```
auto first = std::make_shared<Texture>("hero.png");  
auto second = first; // 올바른 공유
```

```
Texture* raw = first.get();  
// std::shared_ptr<Texture> third(raw); // 잘못된 설계
```

- get() 결과는 비소유 포인터
- 새 스마트 포인터의 소유권 시작점으로 사용하지 않음



7.8 std::weak_ptr

- shared_ptr가 관리하는 객체를 소유하지 않고 관찰
- 약한 포인터는 객체 수명을 연장하지 않음
- 객체가 소멸했는지 확인 가능
- 실제 사용 전 lock()으로 임시 공유 소유권 획득

```
std::shared_ptr<Texture> texture = std::make_shared<Texture>("hero.png");  
std::weak_ptr<Texture> observer = texture;
```

shared_ptr —————> Texture 강한 소유
weak_ptr -----> Texture 비소유 관찰

```
texture.reset();  
if (observer.expired()) {  
    std::cout << "Texture 소멸\n";  
}
```

활용

- 캐시가 객체 수명을 불필요하게 연장하지 않음
- 관찰자가 대상 소멸 여부 확인
- 공유 객체 사이의 순환 소유 차단
- 비동기 작업이 실행 시점의 생존 여부 확인





- lock()은 살아 있는 객체에 대한 shared_ptr 반환
- 이미 소멸했다면 빈 shared_ptr 반환
- 반환된 shared_ptr가 범위 안에 있는 동안 객체 수명 유지

```
if (std::shared_ptr<Texture> texture = observer.lock())  
{  
    texture->Bind();  
}  
else  
{  
    std::cout << "Texture를 사용할 수 없습니다.\n";  
}  
  
if (auto texture = observer.lock())  
{  
    texture->Bind();  
}
```

lock 성공
↓
임시 shared_ptr 생성
↓
객체 사용 중 수명 유지
↓
블록 끝에서 참조 횟수 감소

- expired() 확인 후 다시 접근하는 방식보다 바로 lock() 권장
- weak_ptr에는 operator->와 역참조 연산자 없음



- 두 객체가 서로를 `shared_ptr`로 소유하면 참조 횟수가 0이 되지 않을 수 있음
- 객체의 수명 책임 방향을 분석해야 함

```
class SceneNode
```

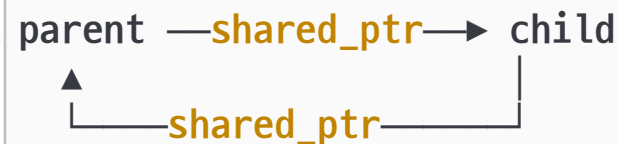
```
{
```

```
public:
```

```
    std::shared_ptr<SceneNode> parent;
```

```
    std::vector<std::shared_ptr<SceneNode>> children;
```

```
};
```



- 외부 소유자가 사라져도 내부 강한 참조가 남음

```
class SceneNode
```

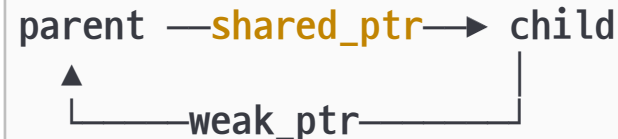
```
{
```

```
public:
```

```
    std::weak_ptr<SceneNode> parent;
```

```
    std::vector<std::shared_ptr<SceneNode>> children;
```

```
};
```



- 부모는 자식을 소유, 자식은 부모를 관찰
- 순환 소유가 아니라 비소유 참조로 표현



- 스마트 포인터의 목적은 원시 포인터 제거가 아님
- 객체 수명을 누가 책임지는지 타입과 구조에 표현
- 동적 할당이 필요하지 않다면 값 객체 우선

소유권이 있는가?

- └ 한 객체만 책임짐 → **unique_ptr**
- └ 여러 객체가 함께 책임짐 → **shared_ptr**
- └ 책임지지 않고 접근함
 - └ 대상이 반드시 존재함 → 참조
 - └ 대상이 없을 수 있음 → 원시 포인터
 - └ **shared_ptr** 객체의 소멸 확인 필요 → **weak_ptr**

표현	소유권	빈 상태	수명 관리
T	값 자체 소유	일반적으로 없음	범위와 포함 객체
T&	비소유	없음	호출자가 보장
T*	비소유로 사용	nullptr 가능	호출자가 보장
unique_ptr<T>	고유 소유	가능	소유자가 해제
shared_ptr<T>	공유 소유	가능	마지막 소유자가 해제
weak_ptr<T>	비소유 관찰	만료 가능	shared 소유자에 따름



- 하나의 객체가 자원 수명을 책임지는 관계
- 수명 종료 시점이 분명
- 참조 횟수 비용 없음
- 복사 금지, 이동으로 소유권 이전
- 합성 관계와 자연스럽게 연결

```
class GameScene
{
private:
    std::vector<std::unique_ptr<Enemy>>
        enemies;
};
```

GameScene — **unique_ptr** —► Enemy

```
std::unique_ptr<Enemy> enemy = CreateEnemy();
scene.AddEnemy(std::move(enemy));
```

- 동적 객체 설계의 기본 선택은 고유 소유권
- 실제 공유 수명이 필요할 때만 공유 소유권 검토

```
class EnemySystem
{
public:
    explicit EnemySystem(GameScene& scene)
        : scene(scene) { }
private:
    GameScene& scene; // 비소유 참조
};
```



- 소유하지 않는 접근에는 참조와 원시 포인터 사용 가능
- 참조: 대상이 반드시 존재해야 하는 관계
- 원시 포인터: 대상이 없을 수 있거나 변경 가능한 비소유 관계

```
class Renderer
{
public:
    explicit Renderer(GraphicsDevice& device) :
        device(device) {}
private:
    GraphicsDevice& device;
};

class Enemy
{
public:
    void SetTarget(Player* target)
    {
        this->target = target;
    }
private:
    Player* target = nullptr;
};
```

```
if (target != nullptr)
{
    target->TakeDamage(attack);
}
```

- 비소유 포인터와 참조의 핵심 조건은 대상 수명
- 구조적으로 보장할 수 없으면 weak_ptr 검토



7.9.5 소유권을 전달하는 함수 인터페이스

목적	인터페이스
새 고유 소유권 반환	<code>std::unique_ptr<Enemy> CreateEnemy();</code>
고유 소유권 받기	<code>void AddEnemy(std::unique_ptr<Enemy> enemy);</code>
공유 소유권 추가	<code>void SetTexture(std::shared_ptr<Texture> texture);</code>
읽기 전용 접근	<code>void Draw(const Texture& texture);</code>
변경 가능한 접근	<code>void Update(Player& player);</code>
선택적 비소유 접근	<code>void SetTarget(Player* target);</code>
스마트 포인터 자체 변경	<code>void ResetEnemy(std::unique_ptr<Enemy>& enemy);</code>

```
void Sprite::SetTexture(std::shared_ptr<Texture> texture)
{
    this->texture = std::move(texture);
}
```

1. 함수가 객체를 소유하거나 저장하는가?
2. 소유한다면 고유한가, 공유하는가?
3. 소유하지 않는다면 대상이 반드시 존재하는가?
4. 대상이 호출 이후에도 보관되는가?
5. 대상 소멸 가능성을 확인해야 하는가?





상황	일반적인 선택
지역에서만 사용하는 객체	값 객체
클래스가 동적 객체를 단독 소유	<code>std::unique_ptr<T></code>
팩토리가 고유 소유권 반환	<code>std::unique_ptr<T></code> 반환
여러 객체가 실제로 수명 공유	<code>std::shared_ptr<T></code>
공유 객체를 비소유로 관찰	<code>std::weak_ptr<T></code>
반드시 존재하는 비소유 접근	<code>T&</code> , <code>const T&</code>
선택적인 비소유 접근	<code>T*</code> , <code>const T*</code>
가변 길이 원소 집합 소유	<code>std::vector<T></code>

피해야 할 습관

- 모든 동적 객체를 습관적으로 `shared_ptr`로 관리
- 소유권 없는 원시 포인터를 `delete`
- `unique_ptr.get()` 결과로 새 스마트 포인터 생성
- 같은 원시 포인터로 여러 `shared_ptr` 생성
- `release()` 반환값을 잃어버림
- `weak_ptr`를 `lock()` 없이 사용할 수 있다고 판단
- 먼저 수명 책임을 정하고 의미에 맞는 타입 선택





7.10 종합 예제: 게임 장면과 텍스처 소유권

- GameScene 은 여러 Sprite 를 고유 소유
- Sprite 는 Texture 를 공유 소유
- TextureCache 는 Texture 를 비 소유 관찰

GameScene → Sprite 고유 소유권
Sprite → Texture 공유 소유권
TextureCache → Texture 비소유 관찰

GameScene
└─
 vector<unique_ptr<Sprite>>
 ↓
 Sprite
 ↓ **shared_ptr**
 Texture
 ↑ weak_ptr
TextureCache

클래스	소유권 역할
GameScene	스프라이트 수명 책임
Sprite	텍스처 수명 공유
TextureCache	살아 있는 텍스처 재사용, 수명 연장 안 함
Texture	실제 리소스 표현

- 직접 delete 없이 각 타입이 자기 소유권만 표현





- 저장 기간은 저장 공간의 유지 기간
- 객체 수명은 초기화 완료부터 소멸 완료까지
- 자동 객체는 범위를 벗어날 때 소멸
- 동적 객체는 소유자가 수명을 끝내야
- 원시 new, delete는 누수, 이중 해제, 댕글링 포인터 위험
- 메모리 외에도 파일, 소켓, 잠금, 그래픽 핸들은 자원 관리 필요
- RAII는 자원 획득을 생성자에, 자원 해제를 소멸자에 연결
- 결정적 소멸은 범위 종료 시점에 자원 정리를 보장
- 스택 퇴감기 중 생성 완료된 자동 객체는 역순으로 소멸
- 생성자에서 예외가 발생하면 객체의 소멸자는 호출되지 않음
- 이미 생성된 멤버 객체는 자동 소멸
- 소멸자는 예외를 밖으로 전달하지 않도록 설계

